



OrBit User Guide

Module 03: The Shadow Git Architecture

PUBLIC DOCUMENTATION — FOR DEVELOPERS USERS

1 What is Shadow Git?

One of the core innovations of OrBit is its **Shadow Git Architecture**. Developers often hesitate to use real-time collaboration tools if they modify their primary `.git` commit history or interfere with standard branch management workflows (`git checkout`, `git rebase`, etc.).

OrBit solves this by maintaining a hidden bare repository located at:

```
<project_root>/orbit/shadow.git
```

Non-Destructive Guarantee

OrBit never modifies your project's main `.git` folder, index, or HEAD pointer. All OrBit tracking happens inside `.orbit/shadow.git` managed by the Rust daemon via the `git2` crate.

2 The Two-Branch Model

Inside every OrBit Shadow Git repository, the daemon maintains two primary logical branches:

main Branch (Team Baseline): Represents the authoritative snapshot agreed upon by all team members. When deltas are pushed or accepted, they update the local `main` pointer.

sub Branch (Local Scratchpad): Represents the developer's current local uncommitted work. It captures real-time working tree snapshots calculated by the `sync-diff` engine.

3 How Shadow Git Works Under the Hood

When you work in your favorite IDE:

```
[Team Baseline] main --> Commit A --> Commit B --> Commit C (Synced)
|
Local Scratch
sub -----> Commit D (Unpushed)
```

Figure 1: Branch Separation in OrBit Shadow Git

1. The file watcher in `crates/local-daemon` detects filesystem events.
2. The daemon creates an in-memory snapshot manifest via `sync-diff::create_snapshot`.
3. If changes are detected, a temporary tree object is written to `.orbit/shadow.git`.
4. When you click **Push**, a delta commit is generated linking `main` to `sub`, signed with your Ed25519 key, and distributed to peers.
5. When remote peers accept the push, their daemon fast-forwards their local `main` branch in `shadow.git` and merges changes into their working directory.

4 Interoperability with Standard Git

Because Shadow Git operates in isolation inside `.orbit/`, you can continue using standard Git tools alongside OrBit:

```
1 # Standard Git commands work completely normally!
2 git status
3 git commit -m "Official release v1.0"
4 git push origin main
```

OrBit's file snapshot engine automatically ignores the `.git` directory and `.orbit` directory, preventing recursive indexing loops and keeping performance fast.